

---

# GÉNÉRATION DE MUSIQUE JAZZ PAR INTELLIGENCE ARTIFICIELLE

---

RAPPORT FINAL

**Théau Margouty**  
Majeur iA  
ESME  
theau.margouty@esme.fr

**Maxime Beaudoin**  
Majeur iA  
ESME  
maxime.beaudoin@esme.fr

21 mars, 2025

## ABSTRACT

Notre projet s'inscrit dans le cadre d'un plus grand projet de recherche qui porte sur la génération de musique par intelligence artificielle, le projet en question met un point d'honneur à reproduire des musiques de jazz manouche dans le style de Stéphane Grappelli et à superposer des voix chantées générées par IA par dessus. Le projet a pour objectif à termes de produire des répliques 3D d'artistes défunts pour redonner vie à leur prestation sur des œuvres originales, mais notre objectif principal sera dans un premier temps de parvenir à entraîner un modèle à générer de la musique de jazz et dans un second temps de reproduire le style d'artistes spécifiques (ici Stéphane Grappelli) dans le but d'y superposer des voix. Si nos travaux portent leurs fruits ils seront repris dans le cadre de ce sujet de recherche.



## Table des matières

|           |                                          |           |
|-----------|------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction</b>                      | <b>3</b>  |
| <b>2</b>  | <b>Problématique</b>                     | <b>3</b>  |
| <b>3</b>  | <b>Solutions Existantes</b>              | <b>3</b>  |
| 3.1       | Modèle pre-entraîné . . . . .            | 3         |
| 3.2       | Modèles de Markov . . . . .              | 4         |
| 3.3       | Réseaux de neurones récurrents . . . . . | 4         |
| 3.4       | Generative adversarial network . . . . . | 5         |
| 3.5       | Convolutional Neural Networks . . . . .  | 6         |
| 3.6       | From Scratch . . . . .                   | 6         |
| 3.7       | Comparaison . . . . .                    | 7         |
| <b>4</b>  | <b>Proposition et Perspective</b>        | <b>7</b>  |
| <b>5</b>  | <b>Synthèse</b>                          | <b>8</b>  |
| <b>6</b>  | <b>Nouvelle piste</b>                    | <b>9</b>  |
| <b>7</b>  | <b>Conception</b>                        | <b>9</b>  |
| <b>8</b>  | <b>Développement</b>                     | <b>13</b> |
| <b>9</b>  | <b>Conclusion du projet</b>              | <b>14</b> |
| <b>10</b> | <b>Figures</b>                           | <b>15</b> |

## 1 Introduction

Notre sujet de projet porte sur la génération de musique par IA, et notre problématique porte spécifiquement sur le jazz en particulier, même si la même stratégie peut en théorie être appliquée à d'autres styles de musique. Nous avons exploré plusieurs stratégies pour la première étape du projet, cette recherche nous a essentiellement servi à comparer les différents moyens d'arriver à nos fins, qu'il s'agisse d'entraîner un nouveau modèle ou bien trouver des modèles pré-entraînés afin de définir laquelle des méthodes répond le mieux à notre besoin. Dans ce contexte, nous avons exploré plusieurs perspectives :

- Parmi les modèles préentraînés nous avons très rapidement entendu parler du modèle opensource appartenant à méta, à savoir audiocraft ou encore musicgen[1], qui s'avérera être la solution la plus directe pour répondre à notre besoin simplement, c'est un modèle préentraîné, l'idéal ici serait donc de l'entraîner sur un dataset de musique de Stéphane Grappelli pour spécialiser le modèle sur le jazz manouche. Audiocraft est considéré comme la référence dans l'état de l'art à ce jour dans ce domaine spécifique (prompt to music). Nous travaillons à ce jour à obtenir des résultats préliminaires.
- Si les résultats avec les modèles précédemment cités ne se montrent pas concluants (ou pour avoir un contrôle total sur notre modèle) nous étudions la possibilité de créer un modèle from scratch et parmi les techniques existantes (HyperGan, LSTM) les réseaux de neurones traditionnels et les réseaux de convolution de type CNN ont l'air de montrer les meilleurs résultats pour la création de piste MIDI par exemple, pour des raisons que nous détailleront davantage dans une section dédiée.

**Keywords** MIDI, Manouche

## 2 Problématique

Comment concevoir une intelligence artificielle capable de générer des compositions de jazz manouche fidèles, en reproduisant les nuances stylistiques propres à un artiste comme Stéphane Grappelli, tout en intégrant des voix générées par IA de manière cohérente ?

Cette tâche implique certains enjeux incontournables. Cela nécessite de déterminer si un modèle sur mesure est nécessaire pour atteindre cet objectif ou si au contraire adapter des modèles existants permet d'atteindre des résultats satisfaisants. Il faut également explorer les stratégies d'entraînement et les approches permettant d'assurer que les compositions générées soient cohérentes et respectent l'authenticité du style. Enfin, il s'agit de définir des critères d'évaluation fiables pour mesurer la fidélité stylistique et la qualité des productions générées.

## 3 Solutions Existantes

Plusieurs approches tout aussi pertinentes peuvent être envisagées pour atteindre cet objectif même si le résultat attendu diffère selon les méthodes. Il est possible de partir de modèles préexistants, qui peuvent être adaptés à un style musical spécifique par un processus de fine-tuning sur un dataset ciblé. Cette solution permet de gagner du temps, mais peut présenter des limitations en termes de personnalisation et de contrôle sur les résultats. À l'inverse, la création d'un modèle sur mesure offre un contrôle total sur les caractéristiques musicales générées, bien qu'elle nécessite davantage de ressources et de compétences techniques. De toute évidence, plusieurs architectures de deep learning différentes permettent d'arriver à nos fins mais certaines représentent un choix plus judicieux pour le traitement de la musique qui est bien spécifique. Enfin, des techniques intermédiaires, telles que le transfert d'apprentissage ou l'utilisation d'architectures hybrides, pourraient également être explorées pour combiner les avantages des deux approches tout en optimisant les performances.

### 3.1 Modèle pre-entraîné

Les modèles préentraînés comme Audiocraft, MusicLM[2], Jukebox[3] ou Riffusion[4] marquent une avancée majeure dans la génération musicale par IA, en s'inspirant directement des concepts derrière les modèles de génération de texte, d'images et même de discours. Leur fonctionnement repose sur des idées similaires à celles des modèles de text-to-speech ou de text-to-image, où une entrée textuelle (ou une autre forme de conditionnement) sert de guide pour créer une sortie cohérente et stylistiquement alignée.

Des modèles comme MusicLM ou Audiocraft suivent le même principe qu'un générateur d'images tel que DALL-E ou Stable Diffusion : un utilisateur fournit un "prompt" ou une description, et le modèle génère un contenu qui correspond au mieux à cette demande. Dans le cas de la musique, cela signifie transformer une instruction textuelle comme "créer

un morceau de jazz manouche au violon avec des influences swing" en une piste audio complète. Cette capacité découle directement des progrès réalisés dans les architectures de modèles pré-entraînés massivement, souvent basées sur des réseaux transformer.

D'un autre côté, des modèles comme Jukebox vont encore plus loin en essayant d'imiter directement des styles ou des artistes spécifiques, ce qui les rapproche des modèles de text-to-speech qui peuvent imiter des voix humaines. Tout comme un modèle TTS s'appuie sur une immense base de données vocale pour apprendre les inflexions et les caractéristiques spécifiques d'une voix, ces modèles musicaux s'entraînent sur d'énormes bibliothèques d'enregistrements musicaux pour capturer les détails stylistiques d'un genre ou d'un artiste. La différence majeure réside dans la dimension temporelle : la musique, comme le langage, doit être cohérente sur une longue durée, un défi que ces modèles abordent avec des couches temporelles adaptées.

Quant à Riffusion, il propose une approche encore plus visuelle en générant des spectrogrammes (voir référence 1), qui sont ensuite transformés en sons. Cela rappelle directement les modèles de génération d'images, où une représentation latente d'un contenu est traduite en une image visuelle. Ici, le spectrogramme est traité comme une image, et la traduction en audio rappelle la manière dont un générateur d'images fonctionne.

Ces modèles s'appuient également sur l'idée d'embeddings multimodaux, une technologie partagée avec les modèles de génération de contenu non musical. Ces embeddings permettent de relier les textes descriptifs à des éléments sonores ou musicaux, un peu comme un générateur de discours relie un texte à des caractéristiques de voix.

En somme, les modèles préentraînés dans la musique tirent directement parti des avancées en TTS<sup>1</sup>, en génération d'images et en modèles de langage. Ils traduisent des concepts linguistiques ou visuels en une nouvelle dimension sonore, offrant ainsi des outils puissants qui ouvrent la voie à une créativité augmentée par l'IA, tout en restant enracinés dans les approches des autres formes de contenu génératif.

### 3.2 Modèles de Markov

Les modèles de Markov offrent une méthode simple et efficace pour créer des séquences musicales notamment pour la génération de suites d'accords. Leur principe est basique : ils prédisent la prochaine note ou accord en fonction des précédents, en se basant sur les probabilités d'enchaînement. Ce type d'approche est parfait pour reproduire des motifs répétitifs ou des transitions typiques, (il est notamment capable de créer des musique en fonction de l'ambiance de l'utilisateur[5]). C'est pourquoi les modèles de Markov sont pertinents, notamment pour modéliser des structures récurrentes comme celles que l'on trouve dans le jazz manouche. Voici un exemple de graphes de modèles de markov (voir référence 2)

Dans notre projet, un modèle de Markov pourrait servir à générer une première ébauche de séquences MIDI. Par exemple, en l'entraînant sur des morceaux de Stéphane Grappelli, on pourrait lui faire apprendre quels accords ou notes suivent naturellement certaines progressions. Cela permettrait de créer des structures musicales cohérentes, en phase avec les règles harmoniques et mélodiques du style.

Cette méthode a le bénéfice d'être rapide et peu gourmande en ressources comparée aux approches basées sur des réseaux neuronaux. C'est une solution relativement simple à mettre en oeuvre avec peu de moyens et idéale pour poser une base. On pourrait s'en servir comme preuve de concept pour des modèles plus complexes.

Mais il faut reconnaître les limites des modèles de Markov. Ils ont du mal à gérer les dépendances à long terme et passent à côté des subtilités et de l'expressivité qu'on trouve dans des styles comme celui de Grappelli. Ils restent assez basiques et peuvent donner des résultats un peu mécaniques si utilisés seuls.

En substance, même si les modèles de Markov ne suffisent pas pour capturer toute la richesse et les nuances du jazz manouche, ils sont très utiles pour démarrer. Ils posent des bases solides que l'on peut ensuite enrichir avec des techniques plus avancées.

### 3.3 Réseaux de neurones récurrents

La génération de musique avec un RNN<sup>2</sup> est très similaire à la génération de texte avec un RNN. Dans le cas du texte, le modèle utilise une séquence initiale pour prédire les mots suivants probable d'une phrase, on va alors relancer le modèle mais sur la séquence qu'il a généré. Cette prédiction est alors répétée pour générer une phrase ou un paragraphe complet. De manière similaire, pour la musique, on commence avec quelques notes, et le modèle RNN prédit la note suivante et le modèle répète le processus avec les notes prédits jusqu'à composer une piste musicale entière.

---

<sup>1</sup>TTS:Text To Speech

<sup>2</sup>RNN:réseaux de neurones récurrents

Dans la plupart des modèles de RNN, on intègre souvent une couche LSTM<sup>3</sup> [6]. L'ajout d'une couches de LSTM[7] permet de répondre au problème de disparition du gradient, une limitation majeure des RNN classiques ou de modèles. En effet, les RNN classiques sont confrontés à des difficultés lorsqu'il s'agit de mémoriser ou d'apprendre des dépendances à long terme. C'est le problème de disparition du gradient ce qui empêche de modifier les poids en fonction d'événements trop anciens. Les unités LSTM résolvent ce problème grâce à une architecture composée de cellules mémoire et de mécanismes de portes d'entrée, de sortie, et d'oubli qui régulent le flux d'informations. Ainsi, un LSTM peut conserver des informations pertinentes sur des périodes prolongées, permettant au modèle de mieux capturer les dépendances complexes longues dans les données.

Pour notre projet il est possible d'entraîner un modèle RNN avec des pistes MIDI, on peut alors représenter les notes de musique par 3 variables, le pitch, step, et la durée. Le pitch représente la fréquence de la note, le step l'écart avec la dernière note et la durée pour le temps que la note est joué

On peut voir avec un petit modèle RNN-LSTM qu'il arrive à approcher la musique d'origine sur le pitch mais pas vraiment sur les autres métrique (voir référence 3 4). La piste MIDI nous conforte d'autant plus sur le manque de cohérence globale du modèle (voir référence 5)

Il est aussi paramétrable comme pour la génération de texte. On peut inclure un paramètre de température qui régule le côté aléatoire du modèle car si on prend toujours les notes les plus probables la musique sera répétitive et peu créative.

Le modèle RNN est assez complet et simple à entraîner, cependant il est un peu basique et manque toujours un peu de contexte macroscopique.

### 3.4 Generative adversarial network

Les GAN<sup>4</sup>, ou réseaux adverses génératifs[8]. Ils sont utilisés par exemple pour la création de visage, ils reposent sur deux modèles qui travaillent ensemble : un générateur, qui essaie de créer des visages, et un discriminateur, qui juge si ces créations semblent authentiques en les comparant à de vraies données. À force de s'entraîner l'un contre l'autre, ils s'améliorent pour produire des résultats de plus en plus convaincants. Le même procédé est alors utilisé pour la musique. Il faut que le générateur crée des sons ou des séquences musicales, pour que le discriminateur soit capable de les comparer et juger si la sortie est acceptable.

Dans notre projet, les GAN pourraient être un vrai atout pour donner plus de réalisme aux morceaux générés. Par exemple, ils pourraient transformer des données MIDI (des fichiers décrivant les notes et rythmes) en sons proches d'un enregistrement live, en passant par des spectrogrammes ou des signaux bruts. L'idée serait de rendre un morceau numérique aussi vivant qu'un bon vieux jazz manouche, avec un violon qui respire et des nuances naturelles.

Ce qui est intéressant avec les GAN, c'est leur capacité à capturer les détails qui font toute la différence : les glissandos, les vibratos, ou encore les dynamiques propres au style de Stéphane Grappelli. Ils sont aussi bons pour recréer les petites variations aléatoires et expressives qui donnent tout leur charme au jazz manouche, rendant les morceaux moins rigides et plus humains.

Cependant ils ne sont pas simples à mettre en place. En effet le problème est qu'il faut entraîner deux modèles au lieu d'un, il faut beaucoup de temps, et de puissance de calcul. Un dataset riche et fidèle au style visé est essentiel pour que les résultats soient à la hauteur. Le plus important est qu'ils doivent évoluer en même temps si le discriminateur est trop performant alors le générateur ne pourra pas progresser, de même si le générateur est plus performant que le discriminateur, il n'évoluera plus et renverra toujours le même résultat. Il faut le voir comme un match compétitif où les joueurs de même niveau s'entraident, mais si l'écart est trop grand aucun des deux ne peut être enrichi de l'échange. Ça rejoint l'importance d'un dataset de bonne qualité car si un des deux modèles trouve un *loophole*<sup>5</sup> dans les données il pourrait abuser de cette différence et ne plus évoluer.

En bref, les GAN offrent une très bonne solution pour rendre nos morceaux numériques plus vivants et réalistes. Mais il faudra relever quelques défis techniques pour les intégrer efficacement.

<sup>3</sup>LSTM : Long Short-Term Memory

<sup>4</sup>GAN:Generative adversarial network

<sup>5</sup>*loophole*:omission, ici: une manière pour le modèle de répondre au problème d'une manière inattendue ( répéter le même accord à intervalle régulier car il gagne plus de points que si l'état créatif est innové avec des accords différents au moins connus)

### 3.5 Convolutional Neural Networks

Les CNN<sup>6</sup>, bien que conçus initialement pour le traitement d'images, se trouvent avoir une application pertinente dans la génération musicale grâce à leur capacité à extraire des motifs complexes à partir des données. Dans le contexte de la musique, les CNN peuvent être utilisés pour analyser et traiter des représentations spectrales ou des séquences MIDI en identifiant des structures récurrentes comme des motifs rythmiques ou harmoniques, un exemple de modèle connu est wavenet[9].

Lorsqu'ils sont appliqués à des données musicales, les CNN fonctionnent en appliquant des filtres convolutifs pour détecter des caractéristiques locales dans les données. Par exemple, dans une piste MIDI, ils peuvent identifier des enchaînements spécifiques de notes, des répétitions rythmiques ou des accords caractéristiques du jazz manouche. De manière similaire, lorsqu'ils travaillent sur des données spectrales (comme un spectrogramme, une représentation visuelle des fréquences d'un signal audio au fil du temps), les CNN sont capables de repérer des textures sonores complexes et des transitions harmoniques, c'est grâce à ces principes que Riffusion obtient de bons résultats (voir référence 6 7).

Dans notre projet, les CNN sont envisagés comme un complément aux modèles LSTM. Là où les LSTM capturent les dépendances temporelles à long terme, les CNN ajoutent une capacité d'analyse locale plus précise, permettant de mieux modéliser les subtilités du style musical. Par exemple, ils pourraient être utilisés pour affiner la modélisation des dynamiques ou des variations microtonales caractéristiques du jeu de Stéphane Grappelli.

En somme, l'intégration des CNN dans notre approche vise à améliorer la génération musicale en offrant une meilleure compréhension des détails locaux tout en complétant l'analyse temporelle fournie par les LSTM. Cette combinaison pourrait ainsi permettre une génération musicale plus fidèle et plus nuancée, répondant aux exigences stylistiques spécifiques du jazz manouche.

### 3.6 From Scratch

Lorsqu'il s'agit de génération de musique par l'intelligence artificielle, les choix du format des données d'entrée sont cruciaux. L'état de l'art privilégie généralement les pistes MIDI plutôt que les fichiers audio au format MP3 pour plusieurs raisons techniques. Ces choix influencent directement les performances des modèles et orientent également les pistes d'exploration futures. Voici une analyse des contraintes et des avantages associés à cette préférence.

Un fichier mp3 est très dense en information, sur une piste de 1m30 et avec un samplerate<sup>7</sup> de 44100hz<sup>8</sup> on atteint les 7M de données. À titre de comparaison une 1920x1080 pixels avec ses composante RGB contient 6M. On a la distinction entre une musique courte de qualité standard et une image en HD.

A cause de la très grande quantité de donnée à traiter pour une simple musique cela requière une machine très puissante, de plus si on décide de réduire la quantité d'information en coupant par échantillon plus petit on perd alors des informations essentielles de contexte sur la musique.

Un dernier point important est la perception humaine des médias sonores. Dans une image, un léger changement de couleur ou de luminosité d'un pixel est souvent imperceptible, à moins d'y prêter une attention particulière et d'être très proche de l'écran. À l'inverse, même un petit artefact sonore ou une distorsion dans une piste audio est immédiatement perceptible, souvent même sans attention consciente. Cela rend le traitement et la génération de musique beaucoup plus exigeants. Voir le résultat d'un petit modèle qui essaie de débruiter une piste audio (voir référence 8)

La première raison de vouloir utiliser le format mp3 serait de capturer la richesse maximale des informations disponibles. On a vu pourquoi il est très difficile d'y arriver que ça soit en terme de ressource et d'échelle. Le format MIDI enregistre les notes jouées en fonction du temps ce qui a l'avantage de représenter la musique très fidèlement sans trop réduire les informations essentielles, de plus avec le format MIDI l'instrument est totalement dissocié de ce qui nous donne une autre marge de manoeuvre dans la création de musique.

Contrairement avec des modèles de classification de l'image où il est simple d'augmenter les données avec des rotations, translation, et autre effet sur l'image. Il est bien plus difficile d'augmenter un dataset de musique [1]. Il existe cependant des Datasets audio comme celui de [MusicCaps](#)[2] Avec une piste audio est une courte description, et aussi le Dataset [MAESTRO](#) qui contient MIDI et fichier son en WAV

<sup>6</sup>CNN:Convolutional Neural Networks

<sup>7</sup>samplerate:fréquence d'échantillonnage

<sup>8</sup>44100hz est un standard pour les CDs .

### 3.7 Comparaison

Une problématique essentielle pour la sélection de notre modèle est de trouver une architecture qui prenne en compte le contexte global de la musique que l'on génère afin de produire un ensemble cohérent. C'est pourquoi le modèle de Markov est moins judicieux que les modèles intégrant l'architecture LSTM, qui se révèle tout à fait appropriée à notre cas d'usage. En effet, cette architecture prend en compte le contexte entourant les séquences de musique que l'on cherche à créer, ce qui constitue un avantage incontournable par rapport à d'autres architectures. C'est pourquoi les modèles tels que le CNN et le RNN sont très intéressants de ce point de vue.

Le dernier élément à prendre en compte dans le choix des modèles c'est la difficulté et la durée d'entraînement. Le GAN même si très prometteur est bien trop difficile à entraîner et pourrait causer gros retards sur l'avancé du projet. Alors que les modèles de types CNN et RNN sont relativement simple à entraîner il faut cependant toujours une bonne machine avec un GPU puissant

Comme le CNN, qui est davantage spécialisé dans le traitement des images et particulièrement adapté à la reconnaissance de motifs dans l'espace. Le RNN, bien qu'efficace pour le traitement séquentiel, présente des limitations dans la gestion de longues dépendances contextuelles, ce qui le rend moins performant que le LSTM dans notre cas.

Les GAN, quant à eux, offrent une approche intéressante pour générer des données réalistes en apprenant à partir d'exemples, et pourraient être explorés pour produire des variations musicales innovantes tout en préservant la cohérence globale, mais ont la contrainte de nécessiter plus de ressources que des modèles tels que le LSTM à entraîner.

## 4 Proposition et Perspective

### Proposition

Dans le cadre de notre projet, trois approches principales seront privilégiées pour générer de la musique de jazz manouche avec une IA. La première approche se concentrera sur l'utilisation de pistes MIDI, une méthode qui garantit un contrôle plus précis sur la structure musicale et une plus grande prévisibilité dans la génération des compositions, ce qui constitue un avantage considérable aux approches fréquentielles qui nécessitent beaucoup de puissance et de données dont nous ne disposons pas. Cette approche reposera sur la combinaison des avantages des réseaux de neurones LSTM et des réseaux de neurones convolutifs (CNN). Les LSTM, particulièrement efficaces pour capturer des séquences temporelles et des dépendances à long terme, seront utilisés dans un premier temps pour modéliser les séquences musicales complexes du jazz manouche. Par la suite, l'objectif est de combiner les LSTM avec des CNN pour extraire des caractéristiques locales et complexes des données musicales, telles que les motifs rythmiques et harmoniques, afin d'améliorer la génération musicale.

La deuxième approche consistera à développer un modèle hybride qui combine l'approche basée sur les notes MIDI avec une approche fréquentielle. Cette stratégie vise à générer un son plus réaliste à partir des pistes MIDI générées, en traitant les informations musicales au niveau spectral (fréquences) afin d'obtenir un rendu sonore fidèle au style. L'idée serait de combiner un générateur de notes basé sur l'approche MIDI avec un modèle capable de transformer ces notes en un signal audio réaliste. Cela pourrait impliquer l'utilisation de techniques de génération audio basées sur des GAN ou d'autres architectures de modélisation audio, comme les VAE<sup>9</sup>. Cette approche pourrait offrir un rendu sonore plus naturel et fluide, mais elle nécessitera des ressources computationnelles plus importantes et un entraînement plus complexe.

Enfin, la dernière approche se concentrera sur l'intégration des voix générées par IA. Cette méthode nécessitera une étape supplémentaire d'adaptation pour superposer les voix aux compositions instrumentales tout en conservant la cohérence musicale et le style spécifique du jazz manouche. Pour ce faire, des techniques de génération vocale et de synthèse musicale devront être combinées de manière transparente. Cette approche hybride permettra de dépasser les simples limitations instrumentales pour aboutir à des compositions complètes et cohérentes, mais elle représentera un défi supplémentaire en termes de synchronisation et de qualité de la voix générée.

Ainsi, chaque approche présente des avantages distincts, et leur ordre d'implémentation est déterminé par leur complexité technique et les ressources nécessaires à leur mise en œuvre. Nous commencerons par la méthode MIDI, plus simple et plus accessible, avant de développer progressivement les modèles hybrides plus complexes qui combinent données spectrales et génération de voix, dans l'optique d'atteindre des résultats à la fois techniquement solides et artistiquement convaincants.

---

<sup>9</sup>auto-encodeurs variationnels

## Perspective

Nos méthodes sont vouées à évoluer au cours du projet car les méthodes pour générer de la musique sont diverses et éparpillées avec des résultats différents d'une stratégie à l'autre, donc à défaut d'un modèle dominant le secteur de manière fiable pour notre cas d'usage nous explorerons autant de solutions que nécessaires. Nos perspectives incluent à court terme, de prioriser notre attention sur l'entraînement des modèles CNN avec LSTM[10], en tirant parti des forces des CNN pour identifier des motifs dans les spectrogrammes et des LSTM pour capturer les dépendances temporelles complexes. Puis nous expérimenterons avec des architectures GRU<sup>10</sup>, connues pour leur simplicité et leur performance dans les tâches séquentielles, afin de déterminer leur potentiel pour la génération musicale dans le style du jazz manouche.

Dans un second temps, nous explorerons les GAN pour la génération de variations musicales innovantes et stylistiquement pertinentes. Cette approche promet de simuler des interactions plus réalistes entre les instruments, bien qu'elle nécessite une calibration fine et des ressources accrues. Nous travaillerons également sur la création d'un pipeline combinant MIDI et audio, pour générer des compositions exploitant à la fois les aspects symboliques et acoustiques de la musique.

En parallèle l'objectif est de combiner violon et voix, en intégrant des modèles capables de générer une ligne mélodique cohérente pour le violon tout en ajoutant des voix générées par une voix artificielle, fluides et synchronisées avec la musique. L'objectif est de reproduire une expérience fidèle au style.

(voir référence 9)

## 5 Synthèse

En synthèse, notre état de l'art repose sur une exploration des différentes approches et technologies actuelles utilisées pour la génération musicale assistée par IA, particulièrement dans le cadre du jazz manouche. Les modèles de génération musicale peuvent être classés en deux grandes catégories : ceux qui se basent sur des représentations symboliques, comme le format MIDI, et ceux qui traitent directement des données audio ou spectrales.

D'abord, les modèles basés sur les pistes MIDI, notamment ceux qui combinent des LSTM (Long Short-Term Memory) et des CNN (Convolutional Neural Networks), permettent de générer des séquences musicales structurées tout en capturant les dépendances temporelles et en identifiant des motifs rythmiques et harmoniques complexes. Cette approche présente l'avantage d'un contrôle précis sur la structure musicale, ce qui est essentiel pour générer des compositions fidèles au style spécifique du jazz manouche.

Ensuite, l'utilisation de modèles hybrides, qui associent des données MIDI à des informations fréquentielles, permet d'améliorer la génération audio en traitant les données au niveau spectral. Cette approche, qui intègre des techniques comme les GAN (Generative Adversarial Networks) ou les VAE (Variational Autoencoders), est plus complexe mais permet d'obtenir des sons plus réalistes et fluides. L'intégration de ces modèles, en particulier dans la génération audio, est un défi de taille en raison de la nécessité d'un entraînement plus poussé et de ressources computationnelles importantes.

Enfin, l'intégration de voix générées par IA représente une étape supplémentaire, visant à superposer des voix chantées aux compositions instrumentales. Ce processus implique des défis liés à la synchronisation et à la cohérence musicale entre les instruments et les voix générées, mais il permet d'aboutir à des compositions complètes et authentiques.

L'implémentation de ces approches devra se faire de manière progressive, en partant de la génération de pistes MIDI, puis en explorant l'amélioration audio et l'intégration des voix. Chaque étape permettra de poser les jalons nécessaires pour la conception d'une IA capable de reproduire fidèlement le style du jazz manouche en offrant une expérience musicale augmentée grâce à la synthèse audio.

---

<sup>10</sup>Gated Recurrent Units

## 6 Nouvelle piste

### Veille Scientifique

Dans cette section nous allons brièvement expliquer les nouvelles pistes explorées et découvertes après notre soutenance de mi-parcours. Après état de l'art initial nous avons évidemment continué de nous tenir informé des dernières technologies mises en œuvre dans notre domaine.

C'est ainsi que nous avons pris connaissance de deepseek. Malgré que la génération de texte soit très éloigné à notre sujet nous nous sommes quand même intéressé car intégrer un petit modèle comme deepseek à notre projet apporterai des bénéfices significatifs en terme d'expérience utilisateur ou d'ergonomie. En parallèle avons alors constaté et étudié l'utilisation des transformer dans la génération de musique (cf. Google Magenta [roberts2018hierarchical], OpenAi Musenet, et les études de Nathan Fradet sur la musique symbolique [3]), et nous y avons donc apporté une attention particulière à cette solution qui paraissait plus aboutie que d'autres architectures moins développées.

Nous avons décider alors de changer notre agenda pour travailler en parallèle sur une solution de transformer et une autre avec le CNN-LSTM.

## 7 Conception

### MIMIC : Musical Intelligence and Machine-Inspired Composition

Dans cette section, nous allons détailler l'ensemble de nos choix de conception, en expliquant les raisons qui nous ont poussés à les adopter. L'essor récent de l'IA est certes surprenant, mais parvenir à l'intégrer à la musique de manière authentique reste un défi à part entière. C'est pourquoi nous allons expliquer, pas à pas, notre démarche et nos choix technologiques.



MIDI est protocole permettant d'échanger des informations musicales comme les notes, l'intensité, le tempo... En quelque sorte, c'est l'alphabet numérique de la musique. Bien qu'il existe de nombreuses gammes musicales à travers le monde, le protocole MIDI repose sur la gamme chromatique occidentale de 12 notes (do-do#-ré-ré#-mi-fa-fa#-sol-sol#-la-la#-si). Pour préciser, cette gamme de 12 notes correspond à la gamme diatonique de 7 notes que nous connaissons tous, avec en plus les 5 altérations (les touches noires du piano). C'est sur cette base que nous avons travaillé.

### Pourquoi choisir MIDI plutôt qu'un traitement direct de l'audio ?

Créer une IA capable de générer de l'audio entièrement mixé et masterisé ne semble-t-il pas plus intuitif? Oui, mais cela vient avec un certain nombre de contraintes.

La taille des fichiers et la complexité du traitement L'entraînement d'un réseau neuronal sur des données audio brutes est bien plus gourmand en ressources que l'entraînement sur des séquences de notes MIDI. Or, nous ne disposons pas de ressources illimitées.

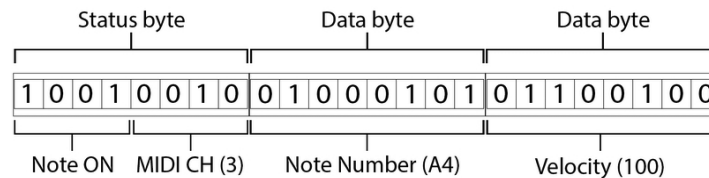
Un outil plus flexible pour les musiciens et créateurs Notre objectif n'est pas simplement de générer des morceaux finis, mais de permettre aux artistes de les modifier et de les intégrer à des compositions existantes. Le choix du MIDI nous permet d'insérer nos créations dans des logiciels audio numériques (DAW – Digital Audio Workstation, e.g. Ableton, FLstudio, LogicPro), ces logiciels utilisés par les compositeurs et producteurs pour peaufiner leurs morceaux.

Avec le MIDI, il devient facile :

- d'ajouter ou de retirer des instruments,
- d'affiner certains passages,
- d'enregistrer une voix humaine pour enrichir la composition,
- voire de créer un duo entre un artiste vivant et un Louis Armstrong numérique jouant de la trompette !

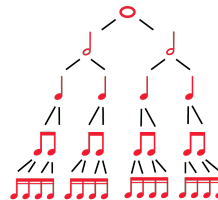
Cette approche s'éloigne du marché grand public, mais elle propose un outil bien plus pertinent pour les créateurs et les musiciens expérimentés. En 2025, Suno est la référence en matière de génération audio basée sur l'IA, produisant des résultats de plus en plus réalistes pour un auditeur non averti. Plutôt que de suivre cette voie, nous avons choisi d'explorer un territoire différent, en offrant une solution unique et plus adaptable dans un cadre de production musicale. Ce projet sera récupéré par notre superviseur dans le but de mener un programme qui vise à faire perdurer la musique d'artistes défunts donc notre ambition est de générer et synthétiser des séquences musicales authentiques. Le choix du MIDI s'impose naturellement : il est à la fois plus performant et plus malléable.

\*la vélocité indique l'intensité de la note



## Miditok

Pour entraîner un modèle de type transformer, nous avons besoin d'une bonne méthode de tokenization. C'est là qu'intervient Miditok [11]. Miditok est un projet lancé par une équipe de chercheur qui a pour mission de tirer parti des récents progrès en traitement du langage naturel (NLP) afin de les utiliser en musique prédictive pour créer une méthode plus efficace de tokenisation qu'une tokenisation par événements musicaux, qui n'est pas optimale. Un événement musical est l'unité élémentaire d'information d'une piste midi, Miditok permet de les regrouper avec des approches plus ou moins simples. Cet outil permet de convertir les événements musicaux des données MIDI (e.g. un changement de vélocité) sous forme de tokens exploitables par un réseau de neurone, un peu comme les tokens utilisés par les large language model (llm). En effet Miditok nous permet non seulement convertir nos données midi dans le but de fournir à nos modèles des données complémentaires au simple format midi (comme la durée de chaque note et sa position dans la Mesure) mais parallèlement à la tokenisation des mots pour les modèles de NLP, Miditok nous ouvre les porte de l'encodage symbolique des piste midi pour les modèles de réseaux de neurones appliqués à la musique prédictive.



L'utilisation de cette librairie vient avec un potentiel créatif non négligeable, en effet pour expliquer notre choix d'utiliser Miditok en des termes simples nous devons faire une petite parenthèse de théorie musicale: une séquence harmonique, mélodique, ou rythmique est de toute évidence rarement représentée avec des notes individuelles, elles sont souvent représenté par paires, ou par 4 (respectivement une croche et double croche) ou souvent 3 pour des accords en musique actuelle. Miditok peut donc concaténer les événements musicaux (ou datapoint) en des ensembles de note cohérents (un accord par exemple) cela réduit la marge d'aléatoire et permet de représenter l'information (ici les séquences de notes) avec les méthode d'encodage les plus populaires. Certaines méthodes sont spécialisées pour ne supporter qu'une seule piste midi (un seul instrument à la fois) et d'autres sont en multi-pistes (l'encodage représente plusieurs instruments).

## Méthodes d'encodage supportées par Miditok

### Encodage basé sur le rythme et la hauteur

- **Pitch-Class Encoding** : Représente les différentes hauteurs de notes sans se soucier de leur octave, ce qui permet une abstraction plus élevée des éléments mélodiques.
- **Note-Interval Encoding** : Utilise l'intervalle entre les notes pour encoder les relations harmoniques et mélodiques de manière plus flexible et modulaire.
- **Rhythm-Velocity Encoding** : Cette méthode combine la représentation de la vélocité des notes avec leur placement rythmique, permettant une meilleure gestion de la dynamique dans les compositions générées.

### Encodage événementiel (Event-based encoding)

- **MIDI-Like** : Une approche classique où chaque événement musical est encodé sous forme de tokens similaires aux événements MIDI.
- **REMI (Relative MI)** : Un encodage basé sur des relations temporelles pour améliorer la gestion du rythme et des dynamiques.
- **Structured** : Introduit une structuration plus explicite des événements musicaux pour une meilleure lisibilité et un apprentissage plus efficace.

### Encodage avancé (Advanced encoding)

- **TSD (Tokenized Sequential Data)** : Regroupe plusieurs événements musicaux sous forme de tokens séquentiels pour améliorer la fluidité et la cohérence des compositions.
- **Octuple Encoding** : Divise un événement musical en huit sous-parties, ce qui permet une plus grande granularité dans l'encodage des informations musicales.
- **CP Word (Chord-Position Word Encoding)** : Encode les informations relatives aux accords et à leur position dans une séquence musicale, permettant une meilleure représentation des structures harmoniques.

### Encodage multi-piste (Multi-track encoding)

- **Track-based Tokenization** : Permet de traiter les différentes pistes d'un morceau de manière indépendante, chaque piste étant encodée séparément tout en maintenant l'intégrité de la composition musicale dans son ensemble.

### Hybride multi-piste (Audio et Midi)

- **MuMIDI** : Introduit une approche multi-instrumentale en encodant plusieurs pistes simultanément tout en conservant leur indépendance.
- **MMM (Multi-Track Music Machine)** : Un modèle avancé de tokenisation multi-piste, optimisé pour la génération de musique polyphonique complexe.
- **MIDI Layered Encoding** : Utilise une approche par couches, où chaque couche représente une piste différente ou un élément d'un arrangement musical complexe (par exemple, la batterie, les instruments à vent, etc.).

Miditok supporte des méthodes d'encodage très avancées et qui pour certaines peuvent être personnalisées pour inclure d'autres paramètres (comme la pédale de piano). Il est important de noter que ces méthodes d'encodages sont sans perte donc préserve le degré de précision du support d'origine (sauf indication). L'outil a été développé par le chercheur Nathan Fradet et est devenu une référence en la matière. La valeur ajoutée de Miditok comparé à une tokenisation simple est de pouvoir regrouper plusieurs événements musicaux en symboles et rend accessible plusieurs techniques de tokenisation fiables et dédiées aux modèles autorégressifs. Miditok est aussi compatible avec les méthodes de tokenisation hybride. Le nombre de paramètres inclus dans les tokens en musique prédictive étant important, le choix initial de méthode peut avoir un impact sur l'apprentissage du modèle si on vise un résultat authentique.

Selon la technique de tokenization choisie (TSD, Octuple, CPWord), il est possible de regrouper plusieurs événements musicaux ensembles, en motifs que le modèle va utiliser en tant que token, ce qui améliore la structure musicale et les performances du modèle. L'objectif principal de cette approche est de réduire considérablement la taille du vocabulaire en gardant le même degré de précision et en améliorant la compréhension des règles musicales par le modèle (cohérence rythmique, mélodique, et harmonique), tout en augmentant les possibilités de combinaisons.

Grâce à cette méthode, notre modèle pourra générer des morceaux plus fluides et cohérents, tout en optimisant ses performances.

### Transformer

Pour l'approche des Transformers, nous nous sommes basés sur deux vidéos une de 3blue1brown expliquant le fonctionnement d'un Transformer, plus précisément le mécanisme d'attention Attention in transformers, step-by-step | DL6

Le mécanisme d'attention a pour but d'extraire le contexte d'une phrase, comme c'est le cas dans le fonctionnement de ChatGPT.

Il existe une fenêtre d'attention dans laquelle le modèle est capable de faire des calculs pour associer à chaque token de la phrase les liens qu'ils possèdent entre eux. Cela permet au modèle de dissocier le sens de deux mots identiques en fonction du contexte. Dans les phrases "le pied d'une chaise" et "j'ai mal au pied" le sens des deux mots est différent c'est le mécanisme d'attention qui analyse le contexte de la phrase pour changer la représentation vectorielle du mot.

Dans notre cas de génération de musique nous trouvons cette approche très prometteuse. En effet étant donnée qu'il existe beaucoup moins de notes que de mots le contexte est alors primordial si on veut construire une piste de musique cohérente.

Cependant la différence principale entre la musique et le texte est la notion de temporalité. Une note est jouée à un certain moment pour une certaine durée. Il faut alors que le modèle soit capable de prédire une note quand elle commence et quand elle se termine.

Dans un premier temps, nous avons tenté de créer un tokenizer capable de lier trois informations et de faire fonctionner un modèle comme chatGPT/deepseek mais appliqué à nos tokens de musique.

On parcourt la moitié d'un dataset de fichiers MIDI et on enregistre chaque note avec ses informations temporelles dans un vocabulaire. Ensuite, on teste si le tokenizer est capable d'encoder des musiques issues de la seconde moitié du dataset, pour lesquelles il n'a pas pu créer de vocabulaire. Cette dernière étape permet de voir si notre vocabulaire est suffisant pour représenter l'espace des notes de musique jouables.

Il s'avère que cette manière de faire ne converge pas vers un vocabulaire fini. Nos essais ont montré que la taille du vocabulaire ne cessait de croître (de la dépasser les vocabulaires de chatGPT, Gemini ou autre LLM) même en tronquant le temps à la centième de seconde près.

Cependant étant persuadé que le mécanisme d'attention est une approche intéressante, nous avons décidé d'aller plus loin et de créer notre propre transformer. Il nous fallait alors trouver comment associer les trois informations (note, début et fin) car elles sont indissociables. Nous avons connaissance du fonctionnement large des LLM et des transformers. Nous savions qu'il est possible d'entraîner un transformer sur plus d'une donnée d'entrée mais il nous est nécessaire de savoir comment procéder.

Pour cela nous nous sommes documentés principalement sur cette vidéo de Andrej Karpathy sur la création d'un transformer type chatGPT : Let's build GPT: from scratch, in code, spelled out.

Nous avons récupéré le code présent dans la vidéo pour le modifier à nos besoins. La vidéo mentionne notamment l'article de recherche *attention is all you need* [12] qui est à la base un modèle pour de la traduction de texte.

Il est expliqué le fonctionnement de la cross-attention. Le fonctionnement classique de l'attention est la self-attention, qui récupère du contexte sur sa propre séquence d'entrée. La cross-attention va prendre du contexte lié à une autre séquence pour enrichir la génération.

C'est de là qu'est venue notre idée, de créer un modèle qui va générer une suite de notes, qui servira aussi de contexte pour la génération des données temporelles.

Le papier nous propose un schéma du transformer avec un bloc de self-attention et un autre de cross attention (voir référence 10).

Nous avons alors réfléchi à la manière d'adapter cette structure à nos besoins et avons conçu deux architectures :

La première que nous avons appelée **Lazy Connection**. Elle est constituée de trois blocs distincts. Le premier bloc génère une suite de notes sans contexte autre que lui-même, le second prend en contexte lui-même et la séquence de notes pour produire à quel temps la note est jouée, le dernier prend en contexte lui-même et la séquence de temps générée par le bloc précédent pour produire à quel temps la note est interrompue. (voir référence 11)

L'idée de cette structure vient du fait principal que presque n'importe quelle note peut sonner correctement si elle est jouée au bon temps. Ça permet d'avoir un modèle relativement léger, bien qu'il manque de contexte.

La seconde structure nous l'avons appelée **Complex Connection**. Dans cette structure les blocs sont identiques constitués d'un mécanisme de self-attention et deux mécanismes de cross-attention. On a alors chaque donnée qui est capable de prendre le contexte de toutes les autres informations disponibles. Le principal défaut de cette méthode est la taille du modèle qui est déjà grande à cause de la présence de trois transformers, et l'ajout d'autant plus avec tous les liens. Un calcul grossier nous donne deux fois plus de paramètres pour les mêmes hyper-paramètres choisis que le modèle lazy.

## 8 Développement

[dataset adl piano] Nous utilisons le dataset adl piano, pour avoir des pistes relativement simple, nous avons 10k fichier midi, il est impossible d'avoir de l'overfitting dans notre cas.

Pour organiser un bon développement on a établi des étapes à suivre pour nous assurer de nos résultats et des changements des hyperparamètres, car un entraînement prend 1h20 à se compléter.

-On note les hyperparamètres.

-On note les pertes de chaque entrée (note, début, fin).

-On écoute le résultat sur un logiciel capable d'interpréter le fichier midi généré avec l'instrument Grand Piano.

La méthode Lazy connection nous a donné des résultats peu satisfaisants, le modèle n'était pas capable de comprendre les données d'entraînement. Même si la perte est plutôt basse avec une valeur qui approche les 2.2 pour les différentes entrées du modèle. La réalité est que la sortie de la génération n'était pas de la musique semblable au dataset. Le modèle rencontre de grandes difficultés à comprendre les données temporelles : les valeurs de début et de fin ne respectent pas une règle élémentaire du dataset. La première règle basique c'est qu'une note ne commence après qu'elle soit finie, c'est à dire que la valeur de fin<sup>11</sup> doit toujours être strictement supérieure de la valeur de début<sup>12</sup>, ce qui n'est pas le cas. De plus ce modèle a tendance à exagérer la fin, les valeurs sont très souvent absurdes (ce qui amène la musique générée à être entre 10 et 15min alors que le dataset possède très peu de musique si long). Le dernier défaut majeur du modèle est qu'il place une note alors qu'elle est déjà présente et pas encore terminée (alors qu'elle est toujours dans la fenêtre d'attention), c'est à dire qu'il ne comprend pas la durée de la note ni le concept de superposition.

Néanmoins nous avons peu d'attente sur cette configuration du fait du manque de contexte. Cependant nous étions étonnés que le modèle ne soit pas capable d'imiter certains accords ni suite de notes.

La méthode Complex connection ne fonctionne pas pour une toute autre raison. Le comportement d'abord prometteur de la perte (voir référence 13) qui est proche de 1 nous a d'abord encouragé malgré la diminution en temps de la perte en peu étrange. La génération n'était en fait que d'une seule et unique note, le modèle à chaque génération différente choisait une note au hasard (en tous cas il est impossible de savoir si ce n'est pas du hasard dans ce cas précis). Au début pour ne pas avoir ce cas précis nous arrêtons l'entraînement du modèle avant cette seconde chute de perte, ce qui nous a donné des résultats similaires à la méthode lazy. C'est bien plus tard quand nous avons modifié une partie de la structure du modèle que nous avons détecté d'où l'erreur venait. Il s'agit d'une partie que nous avons pas encore expliquée dans le mécanisme d'attention.

Lors de la phase d'entraînement le modèle de transformer style ChatGPT en plus de s'entraîner sur les tokens targets il va aussi s'entraîner sur son propre prompt en prédisant tous les tokens qui ont une entrée, mais pour ce faire on doit masquer les tokens futurs pour pas que le modèle "triche" car quand il sera en mode génération il n'aura pas la possibilité de déduire du contexte des mots suivants. On appelle ça le mask, pour un modèle de traduction[12] le mécanisme de cross attention n'a pas besoin d'être masqué car en effet la phrase à traduire doit être comprise d'en son ensemble car l'agencement des verbes, sujets, compléments change en fonction des langues.

Nous supposons que dans notre cas il ne fallait pas masquer nos blocs car une fois qu'une séquence est produite (que ce soit note, début ou fin) elle fournit un contexte d'ensemble et nous voyons mal où le transformer pouvait tricher, mais à la fin quand même (certainement un fois que l'input de note dans son contexte a une donnée temporelle et que celle-ci lui renvoie elle est très probablement toujours avec une grande influence de l'input de note de départ). Cependant même après avoir corrigé ce bug le modèle ne donne toujours pas de bons résultats.

Une des principales suppositions que nous avons faites est que le modèle sera capable de classifier le temps. En effet pour transmettre les données temporelles nous prenons l'ensemble des temps possibles entre 0s et 999.9s. L'idée c'est qu'avec le contexte des notes la probabilité de prendre une note courte ou longue devrait se faire, mais ce n'est pas ce qu'on remarque. Le contexte n'apporte pas plus d'information que les données temporelles peuvent être utilisées. Un autre essai était de modifier la donnée temporelle de fin en donnée de durée soit durée = fin - début, On évite les problèmes tels que les notes trop longues et aussi la superposition (même si ce point n'est peut-être pas réellement résolu c'est à dire qu'il est moins probable qu'il y ait superposition si les notes sont moins longues) mais malgré ça le résultat semble être comme si on avait touché des notes au hasard complet sur un piano, parfois ça sonne juste puis la suite est une dissonance complète.

Notre seconde supposition est que les notes de musique ont une grande proximité avec le texte. Donc que la génération de l'un et de l'autre ne devait pas exiger de changement d'architecture plus grande. Cependant en plus de la contrainte

<sup>11</sup> fin: le temps en seconde où la note se termine

<sup>12</sup> début: le temps en seconde où la note commence

temporelle, la musique contient bien moins d'informations explicites que le texte écrit. En effet 10 mot( ou même token) peuvent suffire pour exprimer une idée précise alors que 10 notes est suffisant pour exprimer une émotion ou un cadre. Ce qui nous relie a un autre problème qui est la taille de nos modèles, Nous avons utilisé la Z machine de l'ESME disposant d'un GPU de 8GB de mémoire pour notre session, nous limitant à un modèle de 50 millions de paramètres a titre de comparaison GPT3 est a 175 Milliards de paramètre.

Notre modèle n'est pas capable d'avoir une fenêtre d'attention suffisamment grande pour crée un contexte qui apporte suffisamment d'information. nos hyperparamètre donnant les résultats les plus intéressant par rapport au temps était 150 dimension d'embedding, 12 nombre de "heads of attention" 12 nombre de couches ou "layers". La dimension d'embedding correspond a la représentation vectorielle associé a chaque token par le modèle. Chaque dimension encode une signification propre, dans le cas des LLM le mot "homme" et "femme" doivent avoir une dimension en commun qui devrait représenter l'idée d'humain mais par contre il doit aussi y avoir une dimension ou leur valeur sont opposé notamment sur le sexe.. C'est le paramètre le plus important qui a une nécessité d'être très grand pour plusieurs raison, la première et plus la dimension est élevé plus d'information d'instinct sont capable d'être encodé. Dans un second temps un résultat mathématique nous montre que plus le nombre de dimension augmente plus il existe des vecteurs orthogonaux entre eux mais pas de manière linéaire mais plus tôt exponentielle. C'est une raison principale de la bonne "scalability" des modèle type GPT. Dans notre cas on essaie d'avoir le nombre le plus grand possible sans réduire les autres hyperparamètre 150 reste extrêmement faible comparé a GPT3 et ces 12 228 dimension.

Le nombre de "head of attention" représente les "questions" posé au token pour récupérer du contexte. Dans un cas gpt on pourrait imaginer une question comme "est qu'il y a des adjectif qui sont lié à mon token ?" dans la phrase " le petit chat vert" petit et vert vont répondre positivement et alors modifié la représentation vectorielle associé pour y ajouter le contexte.<sup>13</sup> La modification de cette hyperparamètre change moins le nombre de paramètre total, mais augment drastiquement le temps d'entraînement.

Le dernier paramètre important et le nombre de layers. En effet une fois que le mécanisme d'attention a pu modifier le sens du token , ça représentation vectorielle en fonction du contexte, il est enrichie et peu de nouveaux ajouter de nouvelles information aux autres token. C'est pourquoi il y a un nombre de couches de répétition. Pour permettre au modèles de convergé sur un contexte globale très souvent le nombre de "head of attention" et le nombre de layers sont les mêmes pour nous c'est 12 GPT3 utilise 96.

Une fois le développement fini nous avons prévu de réalisé un site internet ou il serait présenté au hasard deux musique. L'utilisateur après avoir écouté les deux échantillons répondra a un quesiton affiché nous en avions prévu deux "qu'elle musique avait vous préféré?" "pour vous qu'elle musique a était généré par IA". Ce qui par la suite nous permet d'établir un classement basé sur le score Elo. Il y aurait de la musique généré par notre modèle, par un modèle de l'état de l'art et de la musique générer par humain. Cependant étant donnée que notre modèle a un niveaux d'une personne qui découvre un instrument pour la première fois et essaie de jouer quelque notes, alors que l'état de l'art permet la confusion entre création humaine et création par IA, nous avons décider que ce n'était pas nécessaire.

## 9 Conclusion du projet

Pour conclure, nous avons réussi à créer un modèle qui génère de la musique et l'approche transformer s'est montrée la plus efficace au vu de ses avantage sur des réseaux de neurones plus simples. Notre ambition était dans un premier temps de réussir à générer des séquences mélodiques, et dans un second temps de réussir à faire perdurer le style de Stéphane Grappéli à travers un fine tuning sur l'œuvre de sa vie. Nous avons exploré de nombreuses possibilités afin de réaliser le modèle le plus performant possible mais nous avons une marge d'amélioration. En effet l'utilisation d'outils de tokenisation performants, et de blocques d'attention, couplées à un GPU avec 8GB de VRAM ne suffisent pas à ce jour pour générer des mélodies convaincantes avec un taux de succès raisonnable. Notre modèle fait 50 millions de paramètres et avec le matériel que nous avons à disposition c'est le modèle le plus puissant que nous réussissons à entraîner, constituer un modèle from sratch était définitivement une approche intéressante à plusieurs fins mais notre méthode bénéficiera grandement d'une puissance de calcul améliorée. A défaut d'une meilleure puissance de calcul il faudrait définitivement privilégier des approches moins gourmandes en ressources qui offriraient des performances satisfaisantes. Les technologies et les ensembles de données disponibles à ce jour permettent difficilement d'obtenir des résultats fidèles à un style en particulier avec du matériel standard dans des temps d'entraînement raisonnable. Mais notre approche a fait ses preuves et méritera un approfondissement futur. Nous n'avons pas tout à fait rempli le cahier des charges que nous visions au départ mais notre veille scientifique continue, au vus des progrès actuels sur les réseaux de neurones des nouvelles méthodes permettront tôt ou tard d'entraîner des meilleurs modèles pour atteindre le niveau de performance souhaité avec des moyens modeste.

<sup>13</sup>le fonctionnement décrit ici est adapté au mot pour permettre une meilleurs compréhension mais dans la réalité il est appliqué au token

## 10 Figures

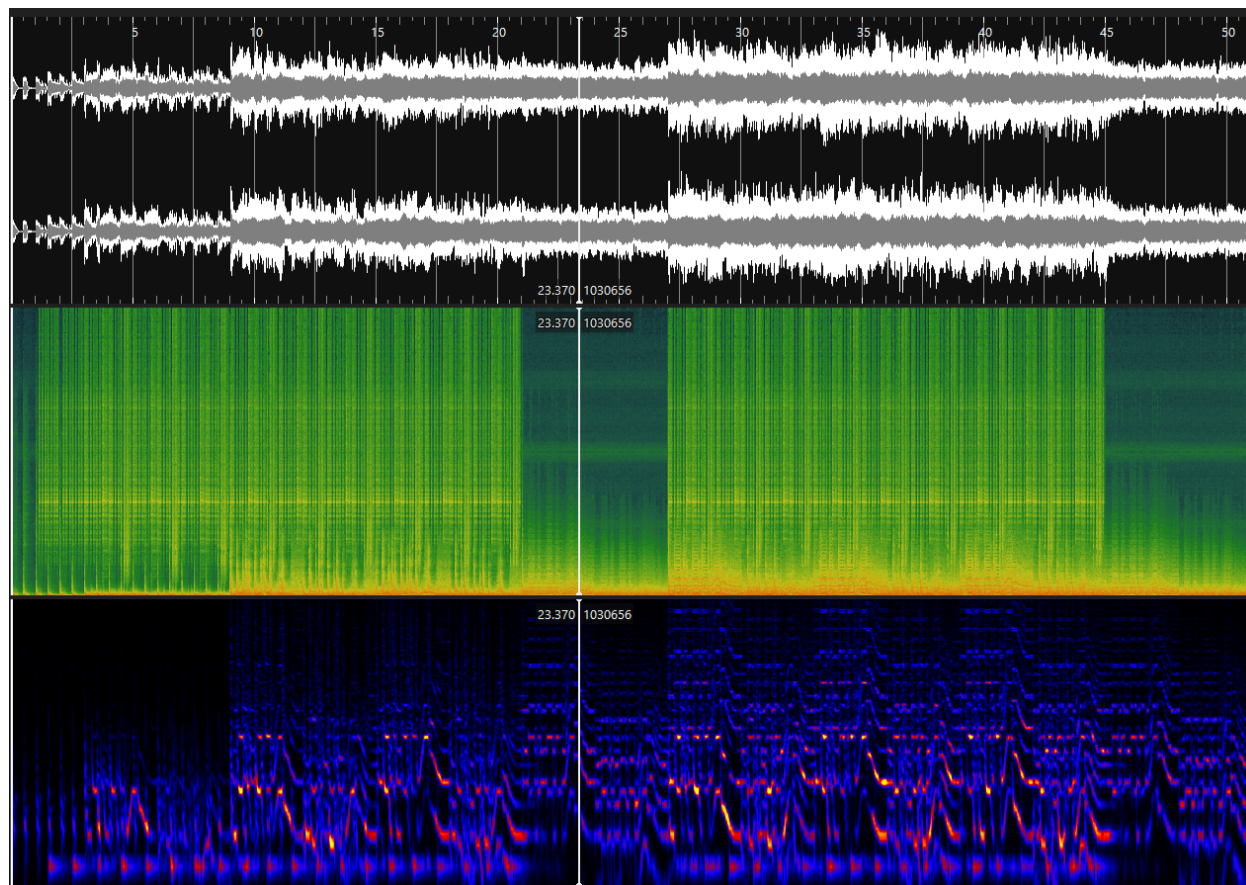
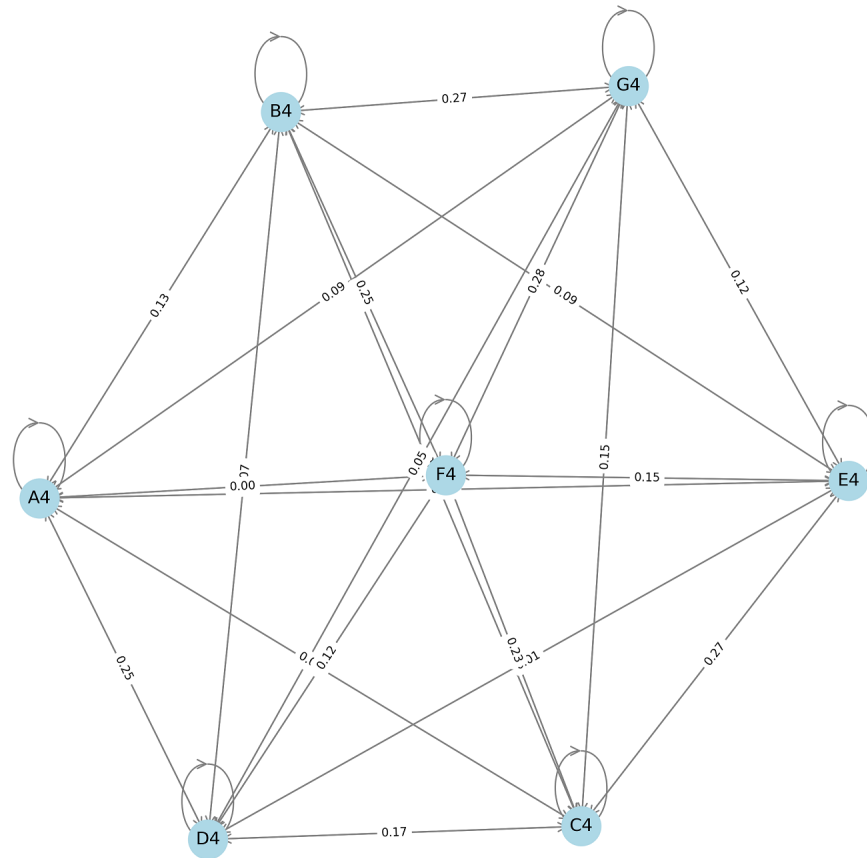
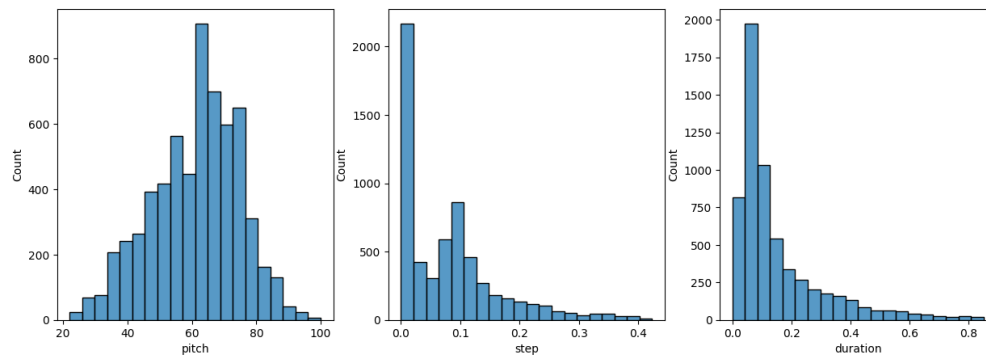


Figure 1: Spectrogram utilisable pour de la modélisation avec un CNN



**Figure 2:** Exemple de graphe d'un modèle de markov pour la génération de musique



**Figure 3:** répartition des valeur de pitch, step, durée pour un sample du dataset

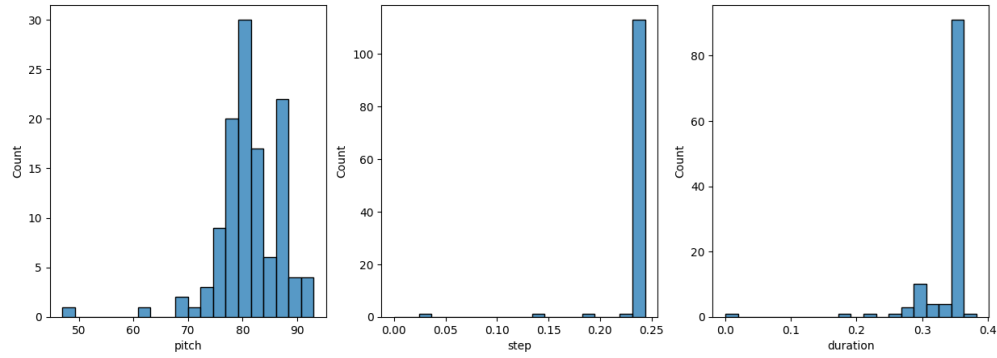


Figure 4: répartition des valeur de pitch, step, durée pour la prédiction du RNN

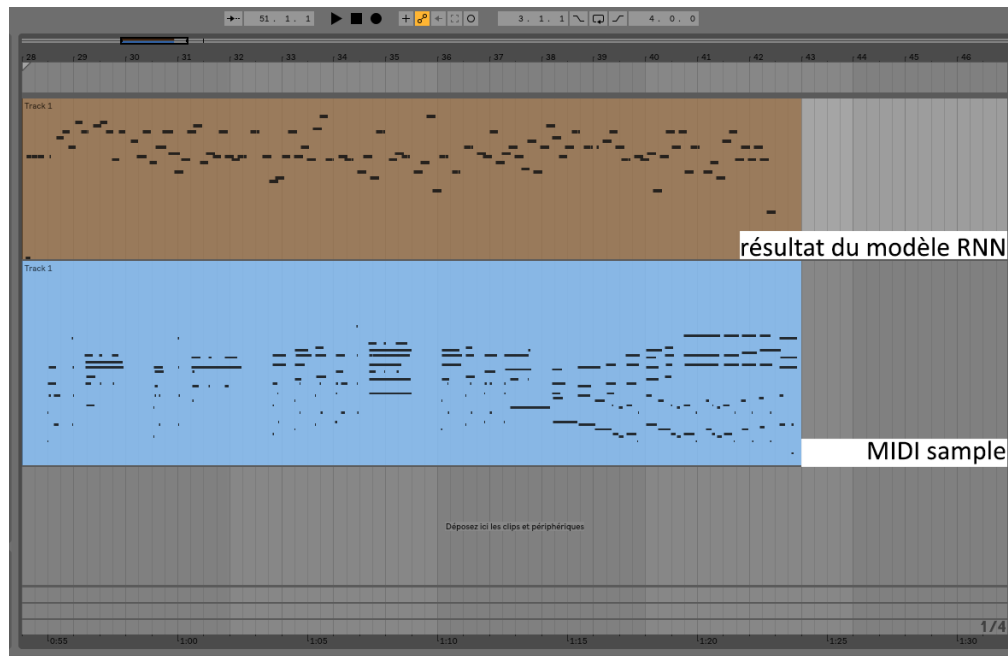


Figure 5: MIDI RNN comparaison (Le sample originale montre que le résultat du RNN est très désordonnée )

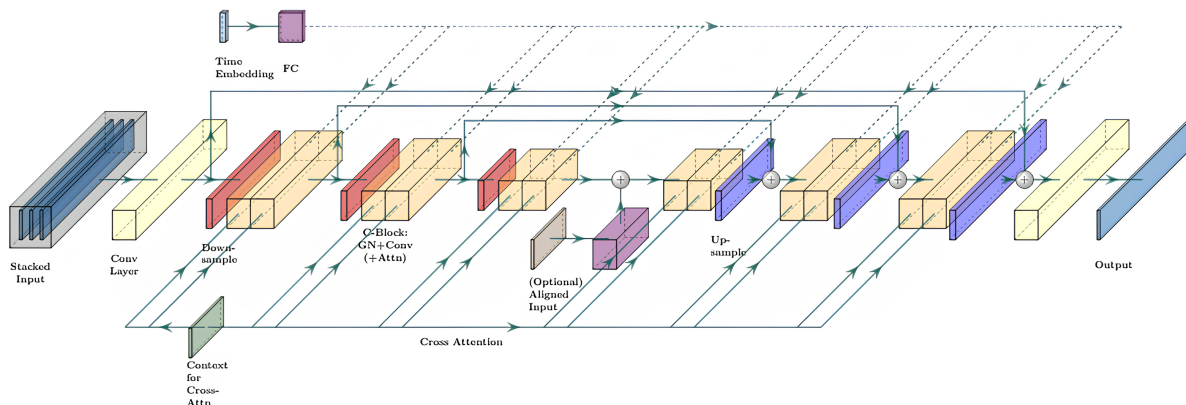
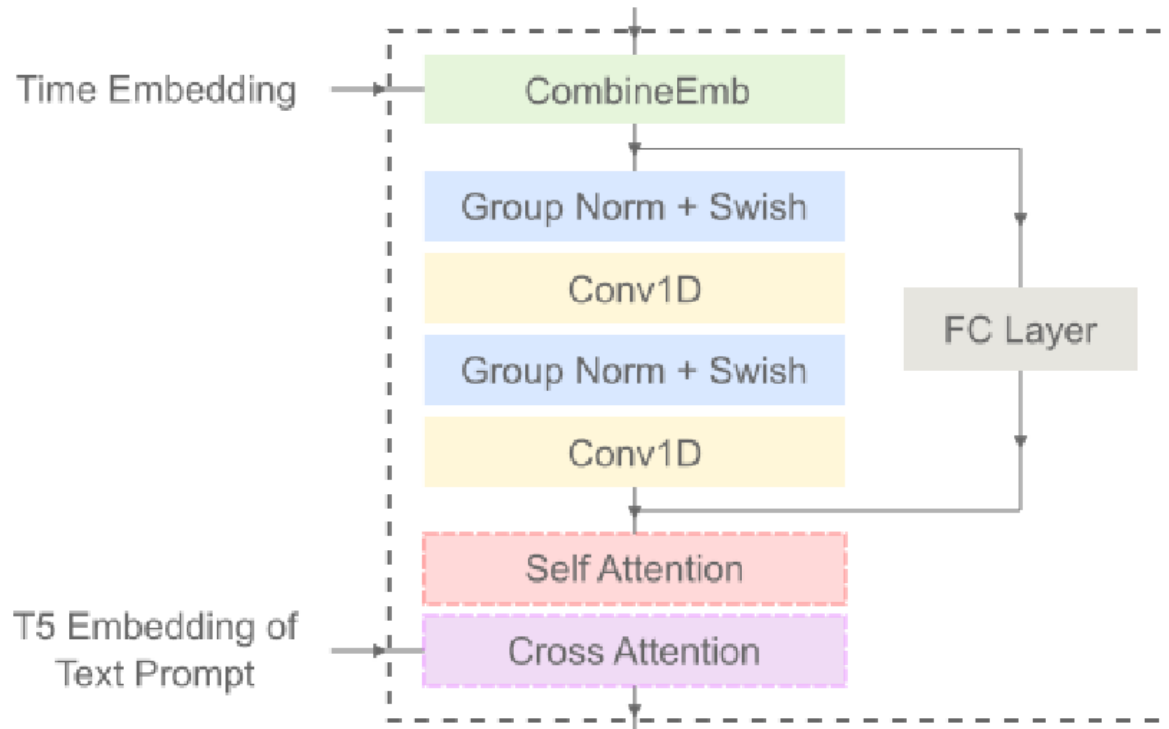
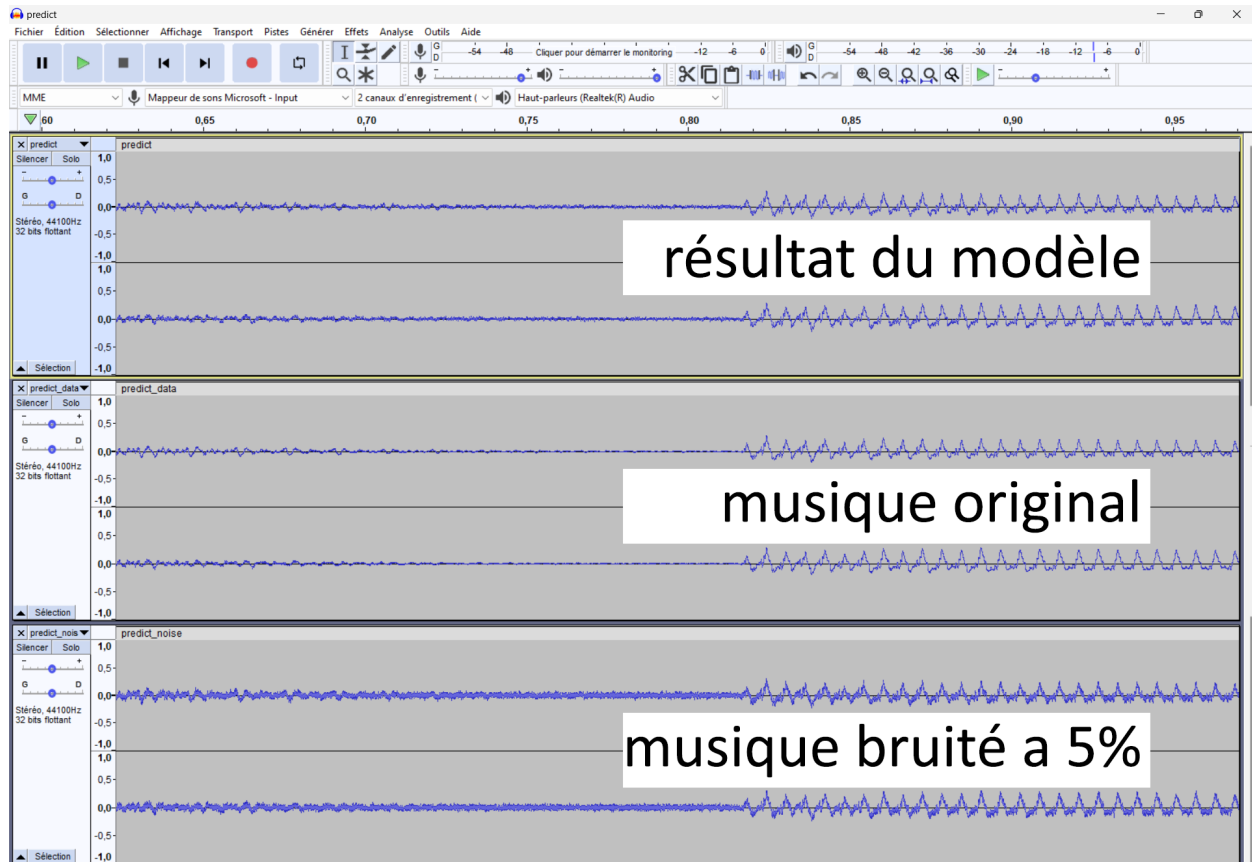


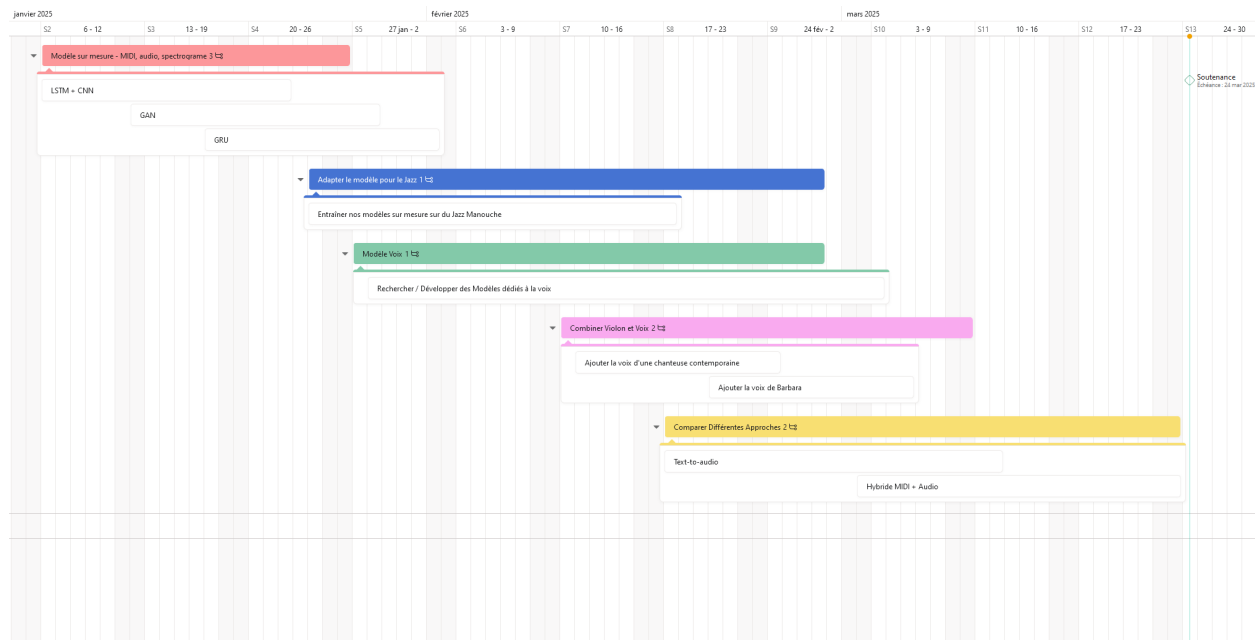
Figure 6: Modèle de CNN avec mécanisme de cross et self-attention et pooling pyramidal pour extraire des caractéristiques à différents champs réceptifs, appliqué à l'architecture U-Net dédiée à l'audio, permettant une meilleure capture des relations temporelles et des détails à différentes échelles.



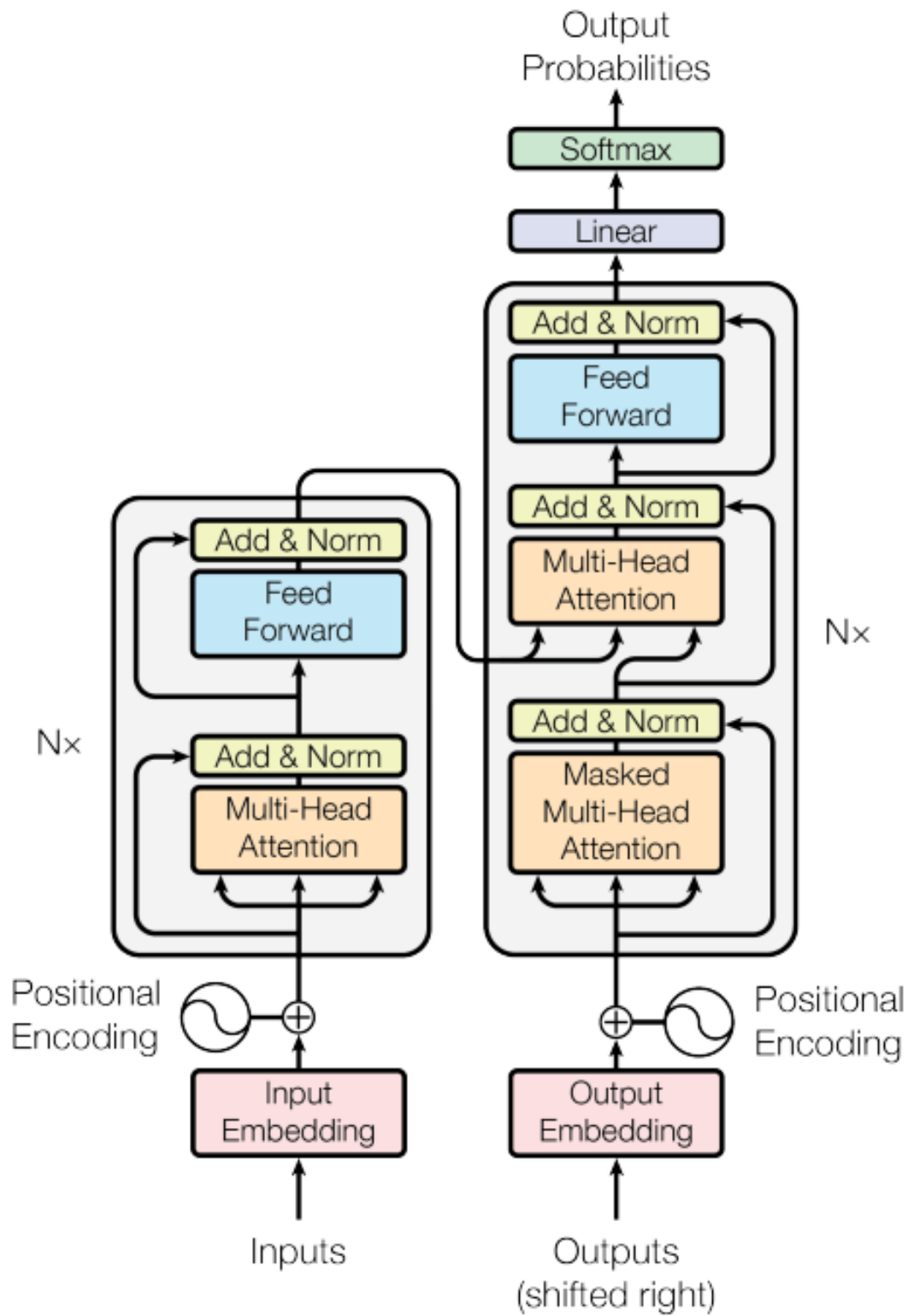
**Figure 7:** Structure d'un bloc de convolution qui forme l'unité de base de fonctionnement dans les U-Nets 1D



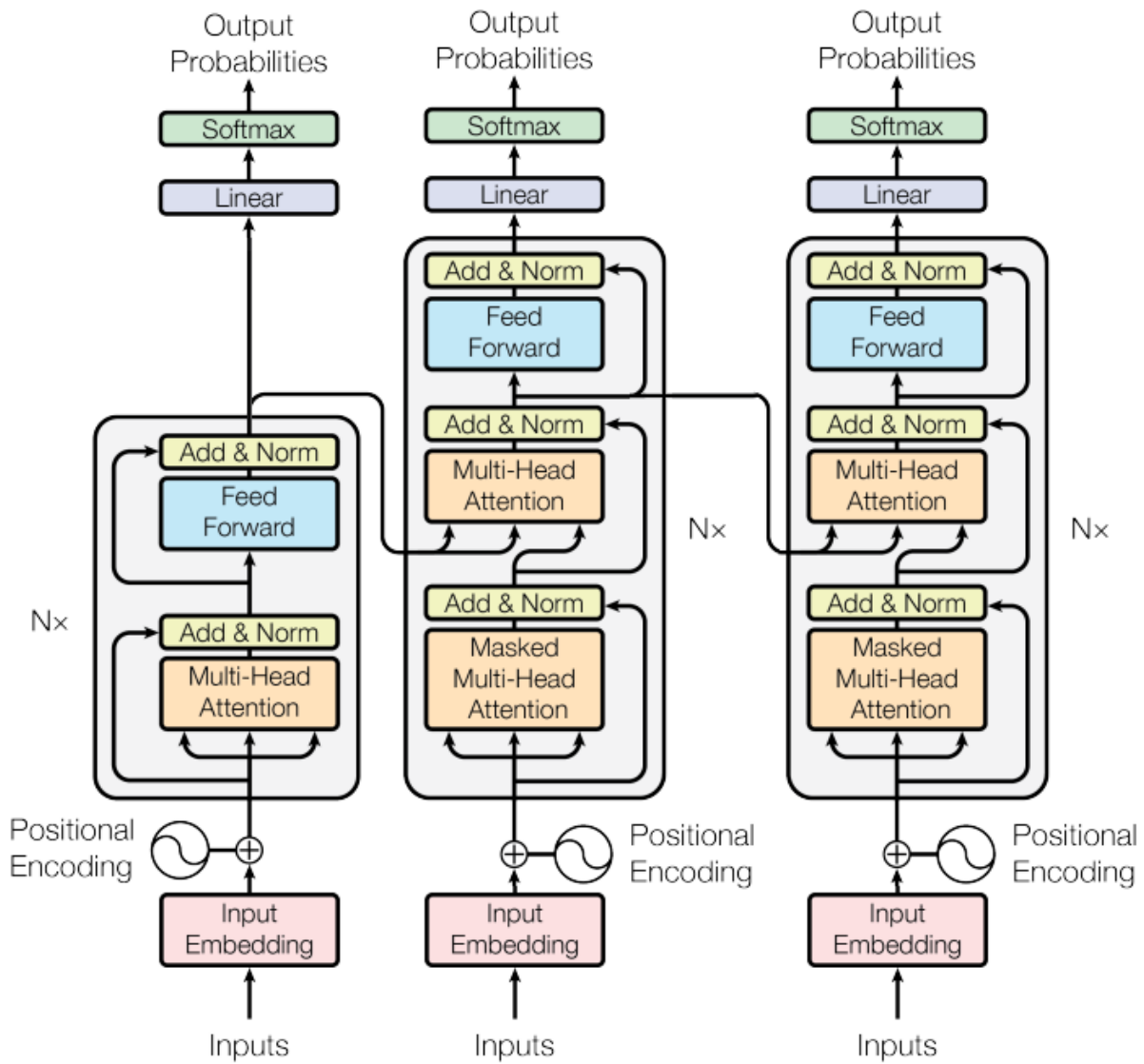
**Figure 8:** musique/bruit/modèle comparaison ( alors que le résultat semble proche de la musique original le son reste très bruité



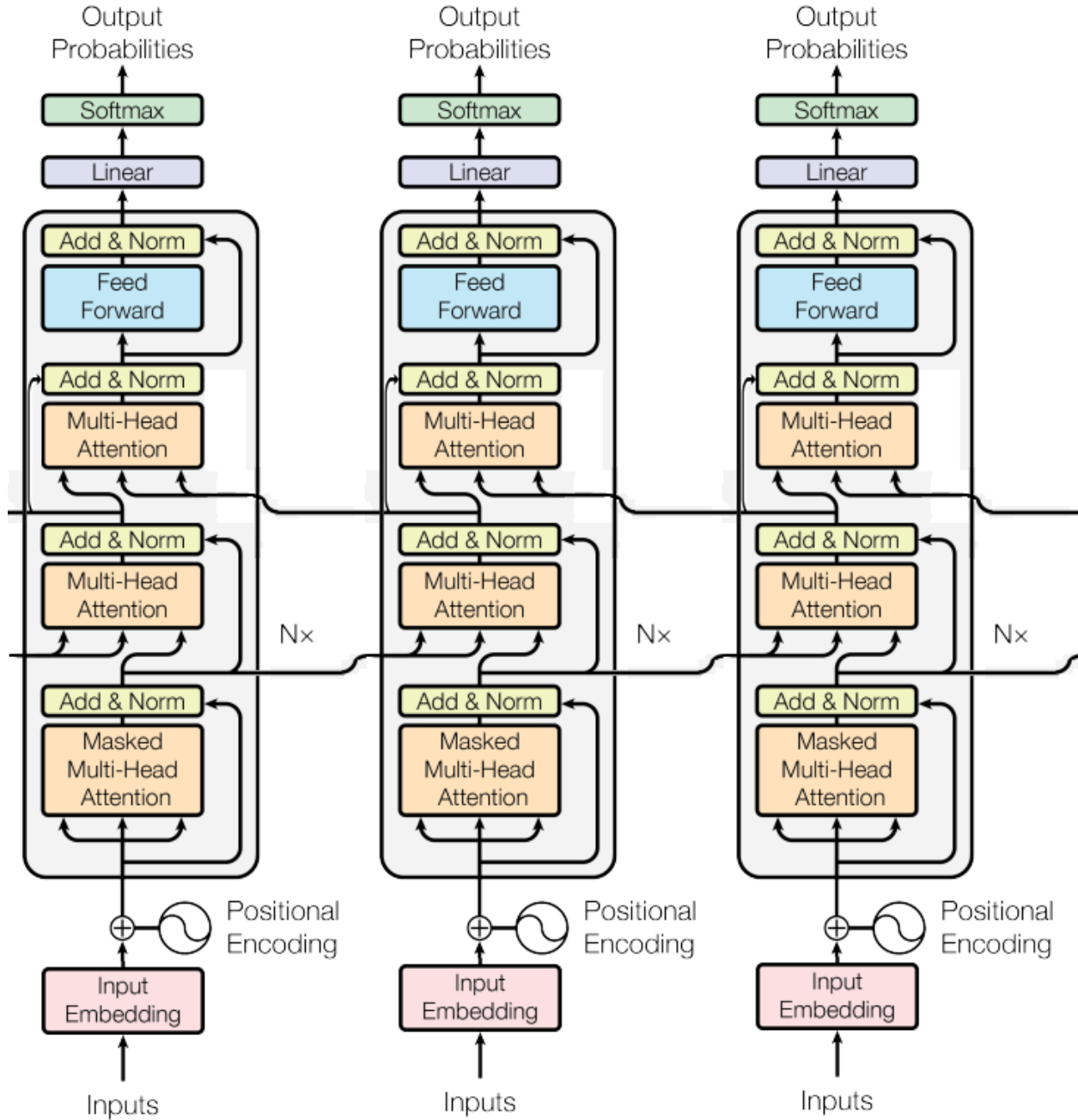
**Figure 9:** diagramme de gantt



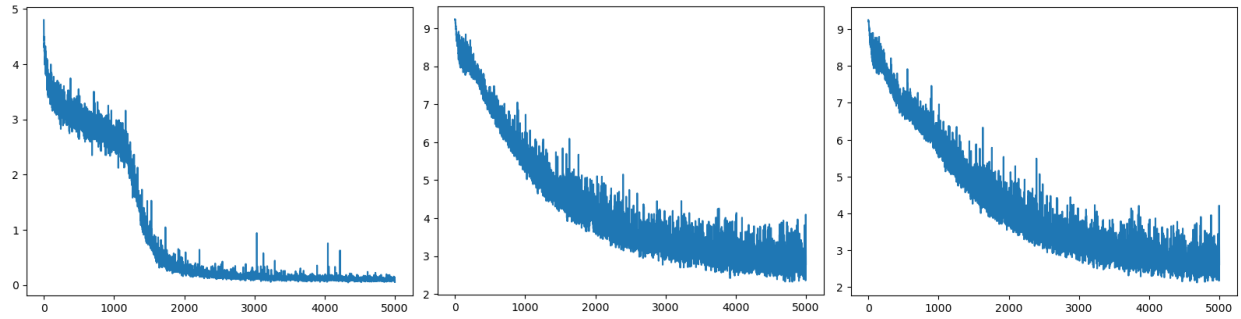
**Figure 10:** schema de transformer du papier "Attention is all you need"



**Figure 11:** schéma de notre transformer en "Lazy connection"



**Figure 12:** schéma de notre transformer en "Complex connection"



**Figure 13:** trois graphique représentant la loss du modèle complex connection avec le nombre d'itération en abscisse et la loss en ordonnée. De gauche a droite les graphs représente la loss de l'input note, début, fin

## Glossaire

**Manouche** La musique manouche ou jazz manouche sont des termes génériques pour désigner un style musical né de la rencontre entre le jazz américain et les traditions musicales des communautés manouches, popularisé notamment par le guitariste Django Reinhardt. Ce genre se caractérise par l'utilisation d'instruments acoustiques tels que la guitare, le violon et la contrebasse, ainsi que par des rythmes swing entraînants, des improvisations virtuoses et une forte influence des musiques gitanes et d'Europe de l'Est.. 3

**Mesure** En théorie musicale, un morceau est divisé en unités rythmiques appelées mesures et délimités par des barres verticales (barres de mesures), elles ont le plus souvent d'une durée fixe de 4 temps, et s'écoulent plus ou moins vite en fonction du bpm.. 10

**MIDI** Musical Instrument Digital Interface dit MIDI est un protocole standard de communication numérique d'échange de données musicales entre différents appareils électroniques (instruments de musique, des ordinateurs, des séquenceurs, logiciels). Le MIDI ne transmet pas des signaux audio, mais plutôt des informations sur les notes, les intensités, les contrôleurs et d'autres paramètres musicaux, c'est analogue à une partition de musique en langage machine dédiés aux logiciels de MAO (digital audio workstation) et aux synthétiseurs.. 3, 9

## References

- [1] Jade Copet et al. “Simple and controllable music generation”. In: *Advances in Neural Information Processing Systems* 36 (2024).
- [2] Andrea Agostinelli et al. “Musiclm: Generating music from text”. In: *arXiv preprint arXiv:2301.11325* (2023).
- [3] Prafulla Dhariwal et al. “Jukebox: A generative model for music”. In: *arXiv preprint arXiv:2005.00341* (2020).
- [4] Seth\* Forsgren and Hayk\* Martiros. *Riffusion - Stable diffusion for real-time music generation*. 2022.
- [5] Adhika Sigit Ramanto and Nur Ulfa Maulidevi. “Markov chain based procedural music generator with user chosen mood compatibility”. In: *International Journal of Asia Digital Art and Design Association* 21.1 (2017), pp. 19–24.
- [6] Sheetal S Patil et al. “Music Generation Using RNN-LSTM with GRU”. In: *2023 International Conference on Integration of Computational Intelligent System (ICICIS)*. IEEE. 2023, pp. 1–5.
- [7] S Hochreiter. “Long Short-term Memory”. In: *Neural Computation MIT-Press* (1997).
- [8] Ian Goodfellow et al. “Generative adversarial networks”. In: *Communications of the ACM* 63.11 (2020), pp. 139–144.
- [9] Aaron Van Den Oord et al. “Wavenet: A generative model for raw audio”. In: *arXiv preprint arXiv:1609.03499* 12 (2016).
- [10] Senem Tanberk and Dilek Bilgin Tükel. “Style-specific Turkish pop music composition with CNN and LSTM network”. In: *2021 IEEE 19th World Symposium on Applied Machine Intelligence and Informatics (SAMI)*. IEEE. 2021, pp. 000181–000185.
- [11] Nathan Fradet et al. “MidiTok: A python package for MIDI file tokenization”. In: *arXiv preprint arXiv:2310.17202* (2023).
- [12] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).